

AGENDA

1. Prefix Sum

Building Psum array

Answering sum queries

Q1 : sum of even indexed

2. Carry Forward

3. What is subarray?

Print all subarrays.

Q. Given N elements and Q queries. For each query, calculate sum of elements from L to R . (both inclusive)

$$1 \leq N, Q \leq 10^5$$

$arr[] = \langle -3, 6, 2, 4, 5, 2, 8, -9, 3, 1 \rangle$

$Q = 3$

	L	R	solution
→	4	8	9
	3	7	10
	1	3	12

- ① Range sum
- ② Multiple queries

Sum = 9

`int query[5][2]`

`query[q][2]`

row	L	R
0		
1		
2		
3		
4		

Brute Force : For each query Q , we iterate from L idx to R idx and calculate sum of elements

```
function querySum(int arr[], int N,  
int Query[][2], int Q)
```

```
// Go to every query, answer it
```

```
for (i = 0; i < Q; i++) {
```

```
    int L = Query[i][0]
```

```
    int R = Query[i][1]
```

```
    // sum of ele from L to R
```

```
    int sum = 0
```

```
    for (int idx = L; idx <= R; idx++) {
```

```
        sum = sum + arr[idx]
```

```
    }  
    print(sum)
```

$[L \ R] \rightarrow R - L + 1$
 $0 \ N-1 \rightarrow N$

1 Query $\rightarrow O(N)$

TC: $O(QN)$

Q queries $\rightarrow O(QN)$

SC: $O(1)$

1 2 3 4 5 6 7 8 9 10
Scores[10] = 2, 8, 14, 29, 31, 49, 65, 79, 88, 97

Runs scored in 7th over = $65 - 49 = 16$
 $[7 \ 7] = S[7] - S[6]$

Runs scored from 6th - 10th over = $97 - 31$
 $[6 \ - \ 10] = S[10] - S[5]$

Runs scored in 3rd - 6th over = $49 - 8 = 41$
 $[3 \ - \ 6] = S[6] - S[2]$

Runs scored from 4th - 9th over = $88 - 14 = 74$
 $[4 \ - \ 9] = S[9] - S[3]$

Runs scored in 10th over = $97 - 88 = 9$
 $[10 \ 10] = S[10] - S[9]$

$$[L \ R] = S[R] - S[L-1]$$

Runs scored in range $\rightarrow$ subtraction
constant time

Magical array of scores
 $\downarrow$

cumulative Runs made from starting till ith over

① cumulative sum array, sum of any range in constant time

② cumulative sum $\rightarrow$ prefix sum array

Prefix sum

① $arr[N] \rightarrow psum[N]$

② $psum[i] \rightarrow$ sum of all elements from starting (0) till i^{th} index

$$A[5] = \begin{matrix} 0 & 1 & 2 & 3 & 4 \\ [2 & 5 & -1 & 7 & 1] \end{matrix}$$
$$p[5] = \begin{matrix} 0 & 1 & 2 & 3 & 4 \\ [2 & 7 & 6 & 13 & 14] \end{matrix}$$

$$p[2] = \text{sum}(0-2)$$

$$A[6] = \begin{matrix} 0 & 1 & 2 & 3 & 4 & 5 \\ [10 & 32 & 6 & 12 & 20 & 1] \end{matrix}$$
$$p[6] = \begin{matrix} 0 & 1 & 2 & 3 & 4 & 5 \\ [10 & 42 & 48 & 60 & 80 & 81] \end{matrix}$$

$$p[3] = \text{sum}(0-3)$$

Q. Creating p[array

// A, N

int p[N]

$$p[0] = A[0]$$

$$p[1] = A[0] + A[1]$$

$$p[2] = A[0] + A[1] + A[2]$$

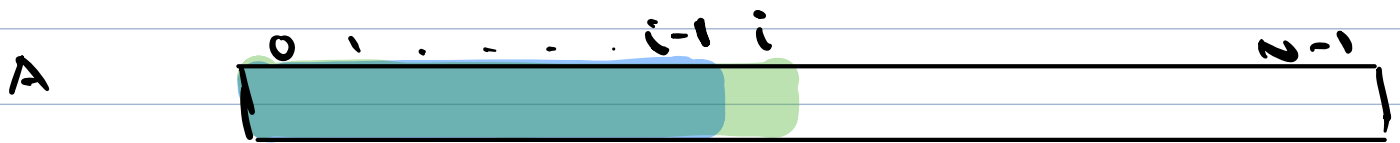
$$\Rightarrow p[2] = p[1] + A[2]$$

$$p[3] = A[0] + A[1] + A[2] + A[3]$$

$$\Rightarrow pf[3] =$$

$$pf[2]$$

$$+ A[3]$$



$$pf[i] = pf[i-1] + A[i]$$

sum(0→i) sum(0→i-1)

int pf[N]

pf[0] = A[0]

TC: O(N)

for (i=1; i < N; i++)

 pf[i] = pf[i-1] + A[i]

A[5] = [2 5 -1 7 1]

pf[5] = [2 7 6 13 14]

$$[L \ R] = S[R] - S[L-1]$$

$$\textcircled{1} \text{ sum}(L-R) = pf[R] - pf[L-1]$$

If L=0

$$\textcircled{2} \text{ sum}(0-R) = pf[R]$$

$arr[] = \langle -3, 6, 2, 4, 5, 2, 8, -9, 3, 1 \rangle$
 $pf[] = \langle -3, 3, 5, 9, 14, 16, 24, 15, 18, 19 \rangle$

$Q = 3$

L	R	solution
4	8	$pf[8] - pf[3] = 18 - 9 = 9$
3	7	$pf[7] - pf[2] = 15 - 5 = 10$
1	3	$pf[3] - pf[0] = 9 - (-3) = 12$

// $arr[]$, N , $Query[][]$, Q

① // calculate $pf[]$

int $pf[N]$

$pf[0] = A[0]$

for ($i = 1$; $i < N$; $i++$) <

| $pf[i] = pf[i-1] + A[i]$

② for ($i = 0$; $i < Q$; $i++$) <

int $L = Query[i][0]$

int $R = Query[i][1]$

// sum of arr from L to R

int $sum = 0$

if ($L == 0$) <

```

    }
    }
    sum = pf[R]
else <
    sum = pf[R] - pf[L-1]
    print(sum)
}

```

TC: $O(N+Q)$
 SC: $O(N)$
 ↓
 pf[]

1 Query $\rightarrow O(1)$
 Q Queries $\rightarrow Q \times O(1) = O(Q)$

SC can be further optimized if we modify the given array

A[5] = [2 ~~5~~ ~~-1~~ ~~7~~ ~~1~~]
 7 6 13 14
 ↓
 pf[]

pf[1]
 ↓
 A[1] = A[0] + A[1]

pf[2] = pf[1] + A[2]
 ↓
 A[2] = A[1] + A[2]

pf[3] = pf[2] + A[3]
 ↓
 A[3] = A[2] + A[3]

for ($i=1$; $i < N$; $i++$) <
 $A[i] = A[i-1] + A[i]$ >
 ↓
 ↘

TC: $O(N)$

SC: $O(1)$

Given N array elements & Q queries, for each query calculate sum of all even indexed elements from L to R . (inclusive)

0 1 2 3 4 5
 $A[i] = 2 \quad 3 \quad 1 \quad 6 \quad 4 \quad 5$

Sum

Queries: 1 3 $A[2] = 1$
 2 5 $A[2] + A[4] = 1 + 4 = 5$
 0 4 $A[0] + A[2] + A[4] = 2 + 1 + 4 = 7$
 3 3 0
 2 2 $A[2] = 1$

① Range sum ② Multiple Queries

BF: For each query, iterate from L to R , if current index is even, add $A[i]$ to sum.

TC: $O(Q \times N)$ SC: $O(1)$

Optimized: Build pf sum

$pf[i] =$ sum of even indexed elements from $0 \rightarrow i$

0 1 2 3 4 5
 $A[i] = 2 \quad 3 \quad 1 \quad 6 \quad 4 \quad 5$
 $pf[i] = 2 \quad 2 \quad 3 \quad 3 \quad 7 \quad 7$

$pf[3] = pf[2]$
 $sc(0-3) \quad sc(0-2)$

$$pf[i] = pf[i-1] + A[i] \quad (\text{odd})$$

$$\text{if } (i \% 2 == 0)$$

$$pf[i] = pf[i-1] + A[i]$$

else

$$pf[i] = pf[i-1]$$

	0	1	2	3	4
$A[5]$	= 2	4	3	1	5
$pf[5]$	= 2	2	5	5	10

	0	1	2	3	4	5
$A[6]$	= 2	3	1	6	4	5
$pf[6]$	= 2	2	3	3	7	7

Queries L R

1 3

$$pf[3] - pf[0] = 3 - 2 = 1$$

2 5

$$pf[5] - pf[1] = 7 - 2 = 5$$

0 4

$$pf[4] = 7$$

$$\text{Sum}(L-R) = pf[R] - pf[L-1]$$

$$\text{Sum}(0-R) = pf[R]$$

// A[], N, Query [][], Q

① // calculate pf[]

int pf[N]

pf[0] = A[0]

for (i = 1; i < N; i++) <

if (i % 2 == 0)

pf[i] = pf[i-1] + A[i]

else

pf[i] = pf[i-1]

> N

② for (i = 0; i < Q; i++) < > Q

int L = Query[i][0]

int R = Query[i][1]

// sum of ele from L to R

int sum = 0

if (L == 0) <

sum = pf[R]

else <

sum = pf[R] - pf[L-1]

print(sum)

TC: $O(N+Q)$

SC: $O(N)$

↓
pf[]

// HW: sum of odd indexed elements

(0:58)

1. Given a string s of lowercase characters, return count of pairs (i, j) such that $i < j$ and $s[i] = 'a'$, $s[j] = 'g'$.

$s =$

0	1	2	3	4	5
a	b	e	g	a	g

ans = 3

i	j
4	5
0	3
0	5

Left Right
 i j
 a g

$s =$

0	1	2	3	4	5	6
a	c	g	d	g	a	g

ans = 4

i j
 0 2
 0 4
 0 6
 5 6

$s =$

0	1	2	3	4	5	6	7
b	c	a	g	g	a	a	g

ans = 5

i, j

i j
 2 3
 2 4
 2 7
 5 7
 6 7

Brute Force : Count how many pairs every 'a' can make ?

For every 'a', count no. of 'g' on right side of 'a'

array of chars
 int countag (string s, int N) {
 int ans = 0
 for (i = 0; i < N; i++) {
 if (s[i] == 'a') {
 for (j = i + 1; j < N; j++) {
 if (s[j] == 'g') {
 ans++
 }
 }
 }
 }
 }

S = a a a a a a

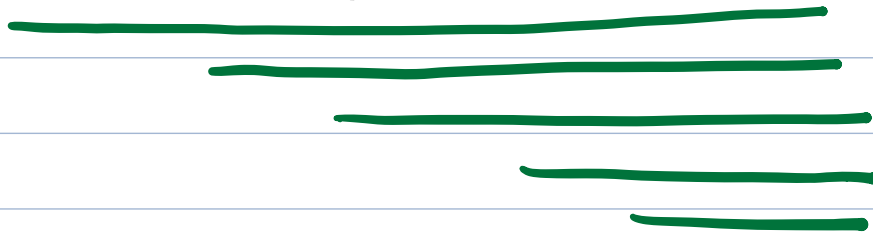
$$[a \ b] = b - a + 1$$

TC: $O(N^2)$

SC: $O(1)$

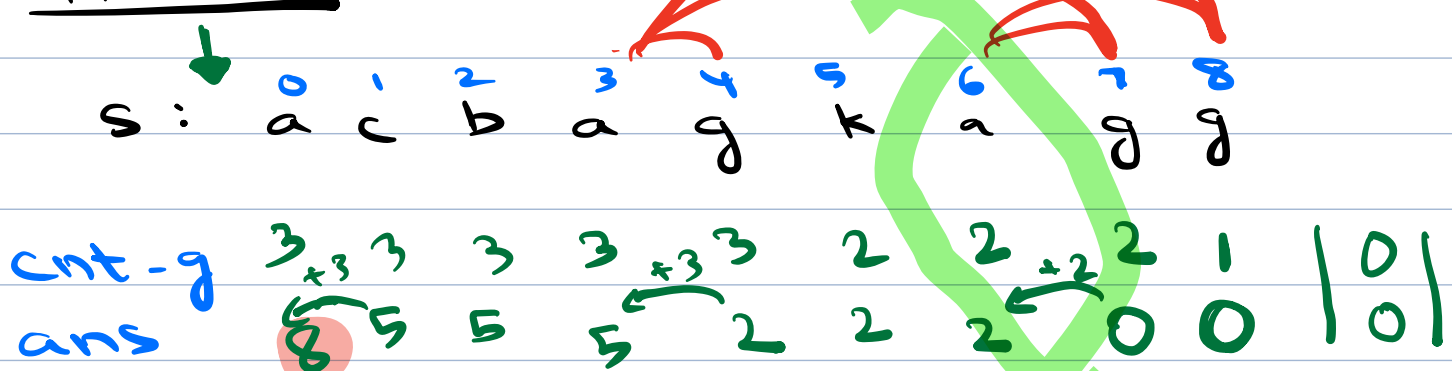
i	j	Iter
0	[1 N-1]	N-1 +
1	[2 N-1]	N-2 +
2	[3 N-1]	N-3 +
⋮	⋮	⋮ +
N-2	[N-1 N-1]	1 +
N-1	[]	0
		$\frac{(N-1)(N)}{2}$

s = a b a a g a a f d c



so in the approach-2 now we are going to count g from right side not from left side as above
so basically we are counting g from right side and increasing the total count and if we found any a then we are making total pair as current available g.

Approach 2 :



① Iterate from right and cnt no. of g's

② Every time we encounter an 'a',

$ans += cnt - g$

so here total pair is 2
as we found 2 g
before any a

Important: the above approach called as carry forward because rather than going sum part of array again and again which are repeating we

- 1) we change the direction of flow
- 2) we are carrying the information forward and keep updating

basically we use carry-forward technique when we have to use the (i,j) pair kind of things

```
int cntag (string s) <
```

```
    int ans = 0 , cntg = 0
```

```
    for (i = N-1 ; i ≥ 0 ; i--) <
```

```
        if (s[i] == 'g') <
```

```
            cntg ++
```

```
        else if (s[i] == 'a') <
```

```
            ans = ans + cntg
```

```
    >
```

```
    return ans
```

```
>
```

TC: $O(N)$

SC: $O(1)$

For every 'g', count no. of a's on left → Brute Force / Carry Forward

Subarrays - It is formed by selecting a range of elements from the array.

It is a contiguous part of array

It can have one or more elements

It should follow order of array

A → ⁰4 ¹1 ²2 ³3 ⁴-1 ⁵6 ⁶9 ⁷8 ⁸12

Examples of subarray

2 3 -1 6
9
3 -1 6

Not subarray

4 1 3 6
4 12
6 -1 3

- ① A single element → subarray
- ② Entire array → subarray

A → 2 4 1 6 -3 7 8 4

<1, 6, 8> X

non-contiguous

<1, 4> X

improper order

<6, 1, 4, 2> X

improper order

<7, 8, 4> ✓

How can we uniquely identify a subarray

- ① start index and end index ✓
- ② start index and length

A : 4 1 2 3 -1 6 9 8 12

Indices: 0 1 2 3 4 5 6 7 8

↓ ↓

2 3 -1 6

< 2 3 -1 6 >

< 9 >

s, e
(2, 5)
(6, 6)

s, l
2, 4

$s \leq e$

Q Subarrays starting at idx 0

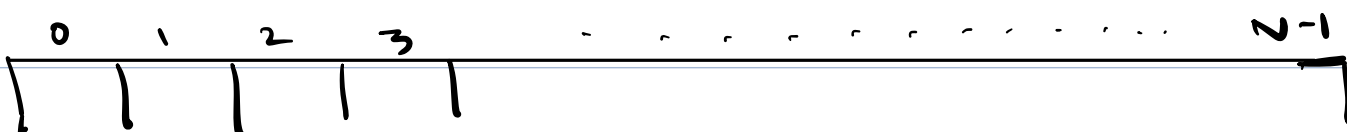
s, e	0	1	2	3	4	5	6	
	4	2	10	3	12	-2	15	ans = 7
0, 0	—							
0, 1	——							
0, 2	————							
0, 3	—————							
0, 4	—————							
0, 5	—————							
0, 6	—————							

Q Subarrays starting at idx 1

	0	1	2	3	4	5	6	
s, e	4	2	10	3	12	-2	15	ans = 6

1, 1	—
1, 2	——
1, 3	————
1, 4	—————
1, 5	—————
1, 6	—————

Count total no. of subarrays in $A[N]$



① Every idx can be 's'

② Count subarrays starting at diff indexes

Subarrays starting at e
idx 0 $\rightarrow$ $[0, N-1] \Rightarrow N$

Subarrays starting at
idx 1 $\rightarrow$ $[1, N-1] \Rightarrow N-1$

Subarrays starting at
idx 2 $\rightarrow$ $[2, N-1] \Rightarrow N-2$

Subarrays starting at
idx $N-1 \rightarrow$

$$[N-1, N-1] \Rightarrow 1$$

$$\text{Total subarrays} = \frac{N(N+1)}{2}$$

A : ⁰4 ¹1 ²2 ³3 ⁴-1 ⁵6 ⁶9 $N=7$

$$\text{Cnt of total subarrays} = \frac{7(8)}{2} = 28$$

